

数据结构机考学习3：树

前两天学习了顺序表和链表，今天开始学习树。

树这一部分和前面两个数据结构有一个比较明显的区别：

顺序表主要靠数组下标，链表主要靠指针指向下一个结点，

而树里面一个结点要指向**左孩子和右孩子**。

我会讲的详细一点点。

一：基础语法：

1.结点定义

```
template<typename T>
struct treeNode {
    T data;
    treeNode<T>* left;
    treeNode<T>* right;

    treeNode(T val):data(val),left(NULL),right(NULL){}
};
```

一个树结点有：

数据域，存储当前结点的数据。

左指针，存储左孩子结点的地址。

右指针，存储右孩子结点的地址。

而当创建一个树结点时，自动给三个成员变量赋初值。

2.私有成员和公有成员

与前面不同的是：

真正递归干活的函数基本都写在 `private` 里面，

比如：

```
pre(root);
inorder(root);
post(root);
level(root);
destroytree(root);
```

而外面调用的时候，

用的是公有函数：

```
preOrder();
inOrder();
postOrder();
levelOrder();
```

这样写的好处是：

外部不需要知道根结点怎么传，只需要直接调用遍历函数即可。

也就是说，

公有函数负责给用户调用，私有函数负责真正实现递归。

这也符合我们那次机考模拟题的风格

二：基本操作

1.完全二叉树创建

方法概述：

这部分是用数组来创建完全二叉树。

```
treenode<T>* creatbyarray(T data[], int i, int n)
{
```

```

    if (i > n || data[i - 1] == 0) return NULL;

    treenode<T>* node = new treenode<T>(data[i - 1]);

    node->left = creatbyarray(data, 2 * i, n);
    node->right = creatbyarray(data, 2 * i + 1, n);

    return node;
}

```

核心思路：

把数组看成一棵完全二叉树。

如果当前结点的位置是 i ，那么：

结点	位置
当前结点	i
左孩子	$2 * i$
右孩子	$2 * i + 1$

注意：

这里的 i 表示的是树中的位置，不是数组下标。

所以：

`data[i - 1]`

才是当前结点对应的数组元素。

2.前序序列创建二叉树

方法概述：

```

treenode<T>* creatbypre(T data[], int& index, int n)
{
    if (index >= n || data[index] == 0)

```

```

    {
        index++;
        return NULL;
    }

    treenode<T>* node = new treenode<T>(data[index]);
    index++;

    node->left = creatbypre(data, index, n);
    node->right = creatbypre(data, index, n);

    return node;
}

```

这部分是根据前序序列创建二叉树。

前序创建的顺序是：

根结点 -> 左子树 -> 右子树

核心思路：

先创建根结点，

然后递归创建左子树，

最后递归创建右子树。

如果遇到 0 ，

就说明这个位置是空结点，

直接返回 NULL 。

注意：

这里的 index 要用引用！！

3.前序遍历

遍历顺序：

根 -> 左 -> 右

代码如下：

```
void pre(treenode<T>* node)
{
    if (node == NULL) return;

    cout << node->data << " ";
    pre(node->left);
    pre(node->right);
}
```

方法概述：

如果当前结点为空，

直接返回。

否则先输出当前结点，

再遍历左子树，

最后遍历右子树。

4.中序遍历

遍历顺序：

左 -> 根 -> 右

代码如下：

```
void inorder(treenode<T>* node)
{
    if (node == NULL) return;

    inorder(node->left);
    cout << node->data << " ";
}
```

```
        inorder(node->right);  
    }
```

方法概述：

先一直访问左子树，

然后输出当前结点，

最后访问右子树。

5.后序遍历

遍历顺序：

左 -> 右 -> 根

代码如下：

```
void post(treenode<T>* node)  
{  
    if (node == NULL) return;  
  
    post(node->left);  
    post(node->right);  
    cout << node->data << " ";  
}
```

方法概述：

先处理左子树，

再处理右子树，

最后处理根结点。

所以后序遍历经常和释放树一起使用。

6.层序遍历

方法概述：

层序遍历就是一层一层输出。

需要用队列实现。

```
void level(treenode<T>* node)
{
    if (node == NULL) return;

    queue<treenode<T>*> q;
    q.push(node);

    while (!q.empty())
    {
        treenode<T>* cur = q.front();
        q.pop();

        cout << cur->data << " ";

        if (cur->left) q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
}
```

核心思路：

先把根结点放进队列。

然后循环：

1. 取出队头结点
2. 输出当前结点
3. 如果有左孩子，把左孩子入队
4. 如果有右孩子，把右孩子入队

直到队列为空。

7.销毁二叉树

```
void destroytree(treenode<T>* node)
{
    if (!node) return;

    destroytree(node->left);
    destroytree(node->right);

    delete node;
}
```

方法概述：

销毁二叉树一般用后序遍历。

因为要先释放左子树和右子树，

最后才能释放根结点。

三：例题

题目1：求二叉树结点个数

思路：

如果当前结点为空，

返回 0。

否则当前结点算 1 个，

再加上左子树结点个数，

再加上右子树结点个数。

```
int countNode(treenode<T>* node)
{
    if (node == NULL) return 0;
```



```
    return 1 + countNode(node->left) + countNode(node->right);  
}
```

核心公式：

结点总数 = 1 + 左子树结点数 + 右子树结点数

比较简单，不再赘述。

题目2：求二叉树高度

思路：

如果当前结点为空，

高度为 0。

否则当前树的高度等于：

左右子树最大高度 + 1

代码如下：

```
int height(treenode<T>* node)  
{  
    if (node == NULL) return 0;  
  
    int leftHeight = height(node->left);  
    int rightHeight = height(node->right);  
  
    return max(leftHeight, rightHeight) + 1;  
}
```

核心公式：

树高 = max(左子树高度, 右子树高度) + 1

题目3：求叶子结点个数

思路：

叶子结点就是没有左孩子，

也没有右孩子的结点。

```
int countLeaf(treenode<T>* node)
{
    if (node == NULL) return 0;

    if (node->left == NULL && node->right == NULL)
        return 1;

    return countLeaf(node->left) + countLeaf(node->right);
}
```

核心判断：

```
node->left == NULL && node->right == NULL
```

只要左右孩子都为空，

说明这个结点就是叶子结点。
